SRI CHANDRASEKHARENDRA SARASWATHI VISWA MAHAVIDYALAYA
(UNIVERSITY ESTABLISHED under section 3 of UGC Act 1956)

ENATHUR, KANCHIPURAM – 631 561

OPERATING SYSTEM LAB

LABORATORY RECORD

Name : Vigneshwar A

Reg. No : 112514023

Class : II Year B.Sc. (Computer Science with Cyber Security)

Subject : BSCY233P60

SRI CHANDRASEKHARENDRA SARASWATHI
VISWA MAHAVIDYALAYA

(University Established under section 3 of UGC Act 1956)

BONAFIDE CERTIFICATE

This is to certify that this is the bonafide record of work done by Mr./~~Ms.~~Vigneshwar A, with Reg.No 112514023 of II Year B.Sc. (CS with Cy) in the Opearting System Lab during the year 2026.

Staff-in-charge **Head of the Department**

Submitted for the Practical Examination held on _____

Internal Examiner **External Examiner**

INDEX

S No.	Date	Title	Page	Sign
1	02Jul2026	Basic Linux Commands	4	
2	09Jul2026	Shell Scripting and Command-Line Arguments	7	
3	23Jul2026	Linux File and Directory Operations Using C System Calls	10	
4	30Jul2026	Process Creation and Management Using Linux System Calls	16	
5	13Aug2026	Signal Handling in Linux	21	
6	20Aug2026	Inter-Process Communication Using Pipes and FIFO	25	
7	10Sep2026	Inter-Process Communication Using Shared Memory	29	
8	17Sep2026	CPU Scheduling Algorithms	34	
9	24Sep2026	Memory Management – Page Replacement Algorithms	39	
10	01Oct2026	Linux File Permissions and File Information	44	

1	Basic Linux Commands	02Jul2026

AIM :

To familiarize students with basic Linux commands for file, directory, and system management, compare them with equivalent DOS commands, and execute them in a Linux environment.

CONTEXT:

This C program demonstrates how to execute Linux shell commands within a C program using the system() function (provided by <stdlib.h> library).

SOME OF THE LINUX COMMANDS:

Command	Function	Example
`pwd`	Display current working directory	`pwd`
`ls`	List files and directories	`ls`
`ls -l`	List files in long format	`ls -l`
`ls -a`	Show all files (including hidden files)	`ls -a`
`cd`	Change directory	`cd Documents`
`mkdir`	Create a new directory	`mkdir Demo`
`rmdir`	Remove an empty directory	`rmdir Demo`
`touch`	Create an empty file	`touch file.txt`
`cp`	Copy files/directories	`cp file1.txt file2.txt`
`mv`	Move or rename a file	`mv old.txt new.txt`
`rm`	Remove a file	`rm file.txt`
`rm -r`	Remove a directory recursively	`rm -r Demo`
`cat`	Display file contents	`cat notes.txt`

SOURCE CODE:

```
#include <stdio.h>
#include <stdlib.h>  //system( )

int main()
{
    printf("Current Directory\n");
    system("pwd");  //to execute an operating system command

    printf("\nFiles\n");
    system("ls -l");

    printf("\nCreating Directory\n");
    system("mkdir Demo");

    printf("\nListing Files Again\n");
    system("ls");

    printf("\nRemoving Directory\n");
    system("rmdir Demo");

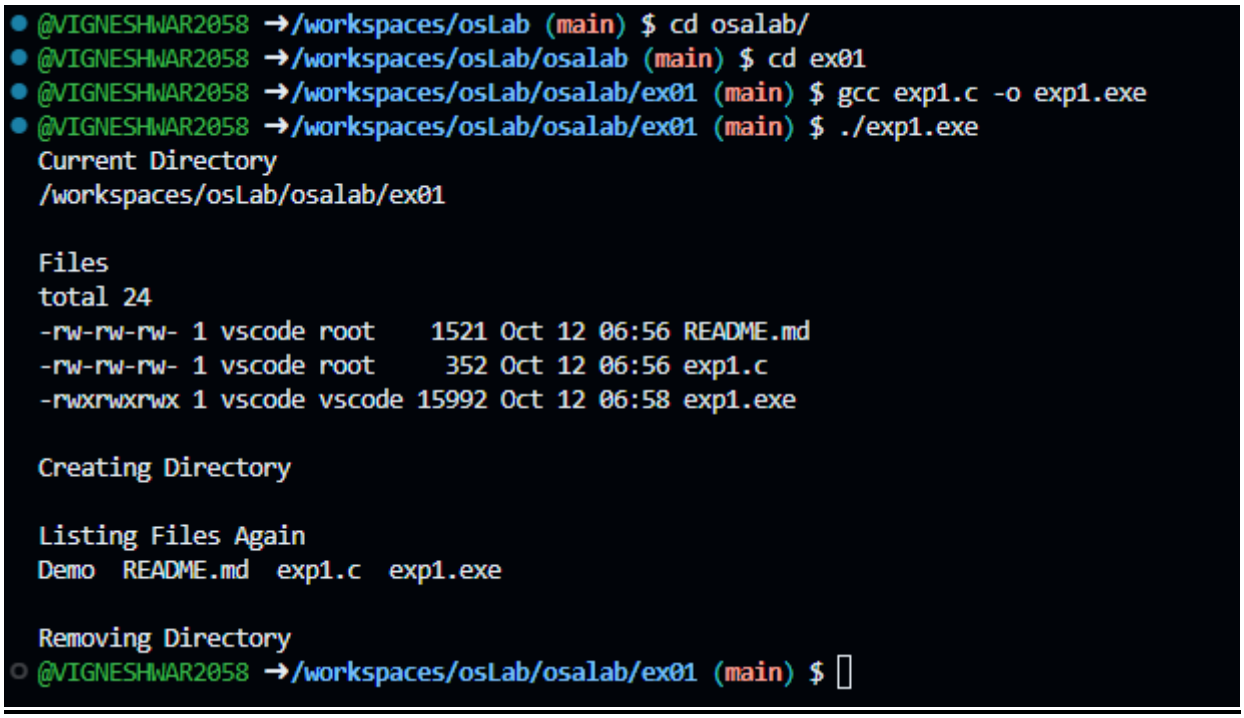
    return 0;
}
```

HOW TO EXECUTE:

Compile (in Terminal): gcc exp1.c -o exp1

Run: ./exp1

OUTPUT :



2	Shell Scripting and Command-Line Arguments	09Jul2026

AIM :

To develop shell scripts for performing basic arithmetic, file handling, decision making, loops, and command-line argument processing using the Bash shell.

CONTEXT:

Shell scripting in Linux is the practice of writing a sequence of commands into a plain text file (extn:sh) to automate repetitive system tasks, configure environments, and manage software deployments. The file is interpreted line by line by a command-line interpreter known as a shell, with the GNU Bourne-Again Shell (Bash) being the most common default version across major Linux distributions.

Commands Used in Your Script

Command/Feature	Function
`echo`	Prints messages to the terminal.
`wc -l`	Counts the number of lines in a file.
`$#`	Returns the number of command-line arguments.
`$@`	Represents all command-line arguments.
`-f`	Checks whether the argument is a regular file.
`-d`	Checks whether the argument is a directory.
`for`	Iterates through each command-line argument.
`if`, `elif`, `else`	Performs conditional execution.

SOURCE CODE :

```
#!/bin/bash
echo "Number of arguments supplied: $#"
for item in "$@"
do
        if [ -f "$item" ]
        then
                echo "$item is a file"
                lines=$(wc -l < "$item")
                echo "Number of lines: $lines"
        elif [ -d "$item" ]
        then
                echo "$item is a directory"
        else
                echo "$item does not exist"
        fi
done
```

HOW TO EXECUTE:

Steps to create:

1. nano checkfile.sh
2. Paste the code given above
3. To save: ctrl+o and press enter
4. To exit: ctrl+x

Execute:

Step 1: chmod u+x checkfile.sh

Step 2: ./checkfile.sh file1.txt testdir sample.txt

OUTPUT :

```
@VIGNESHWAR2058 →/workspaces/osLab/osalab/ex01 (main) $ cd ..
@VIGNESHWAR2058 →/workspaces/osLab/osalab (main) $ cd ex02
@VIGNESHWAR2058 →/workspaces/osLab/osalab/ex02 (main) $ chmod u+x checkfile.sh
@VIGNESHWAR2058 →/workspaces/osLab/osalab/ex02 (main) $ ./checkfile.sh TestDir/ Sample.txt File1.txt
Number of arguments supplied: 3
TestDir/ is a directory
Sample.txt is a file
Number of lines: 2
File1.txt is a file
Number of lines: 6
@VIGNESHWAR2058 →/workspaces/osLab/osalab/ex02 (main) $
```

3	**Linux File and Directory Operations Using C System Calls**	**23Jul2026**

AIM :

To implement basic file and directory operations using Linux system calls such as open(), read(), write(), close(), stat(), mkdir(), opendir(), and readdir().

CONTEXT:

This C program demonstrates the use of Linux system calls for performing file and directory operations.

Linux System Calls Used

System Call	Function
`open()`	Opens an existing file or creates a new file and returns a file descriptor.
`write()`	Writes data from memory to a file.
`read()`	Reads data from a file into a buffer.
`close()`	Closes an opened file descriptor and releases resources.
`stat()`	Retrieves information about a file, such as size, permissions, and number of links.
`mkdir()`	Creates a new directory with the specified permissions.
`opendir()`	Opens a directory stream for reading its contents.
`readdir()`	Reads one directory entry at a time from the opened directory.
`closedir()`	Closes the opened directory stream.

SOURCE CODE :

```
#include <stdio.h>
#include <fcntl.h> // open()
#include <unistd.h> // read(), write(), close()
#include <sys/stat.h> // stat(), mkdir()
#include <dirent.h> // opendir(), readdir()
#include <string.h>
int main()
{
        int fd; //file descriptor (unique id)
        char buffer[100];
        struct stat fileInfo;
        DIR *dir;
        struct dirent *entry;

        // PART 1: Create and write data into a file using open() and write()
        printf("\n--- Creating and Writing File ---\n");
        /*
         * open() system call:
         * Opens/creates "student.txt".
         * O_CREAT: Creates the file if it doesn't exist.
         * O_WRONLY: Opens the file in write-only mode.
         * 0644: Sets permissions to Owner (Read/Write), Group (Read), Others (Read).
         */
        fd = open("student.txt", O_CREAT | O_WRONLY, 0644);

        /* Checks if open() failed (returns -1 on error) */
        if (fd < 0)
        {
                printf("File creation failed\n");
```

```
        return 1;
    }
    char data[] = "Linux System Calls Experiment\n"
          "B.Sc Cyber Security Laboratory";

/* write() system call: Writes data string up to its exact length into the file descriptor */
    write(fd, data, strlen(data));

    /* close() system call: Closes the file descriptor to free system resources */
    close(fd);

    printf("Data written successfully\n");

    // PART 2: Read file contents using open() and read()
    printf("\n--- Reading File Content ---\n");
    /* open() system call: Opens "student.txt" in read-only mode (O_RDONLY) */
    fd = open("student.txt", O_RDONLY);

    /* Checks if the file failed to open */
    if (fd < 0)
    {
        printf("File opening failed\n");
        return 1;
    }
    /*
     * read() system call:
     * Reads up to 99 bytes from fd into 'buffer'.
     * Leaves 1 byte free at the end for the null-terminator.
     */
    int bytes = read(fd, buffer, sizeof(buffer) - 1);
```

```
        /* Safely null-terminates the string at the exact index where reading stopped */
        buffer[bytes] = '\0';
        printf("%s\n", buffer);

        close(fd);

        //PART 3: Display file information using stat()
        printf("\n--- File Information ---\n");
        /*
         * stat() system call:
         * Retrieves file metadata for "student.txt" and stores it inside the 'fileInfo' structure.
         * Returns 0 on success.
         */
        if (stat("student.txt", &fileInfo) == 0)
        {
                /* fileInfo.st_size extracts the total file size in bytes */
                printf("File Size : %ld bytes\n", fileInfo.st_size);

                /* fileInfo.st_nlink extracts the number of hard links pointing to this file */
                printf("Number of Links: %ld\n", fileInfo.st_nlink);

// fileInfo.st_mode combined with bitwise AND (& 0777) isolates the octal permission bits
                printf("Permissions : %o\n", fileInfo.st_mode & 0777);
        }
        else
        {
                printf("Unable to get file information\n");
        }
```

```
        //PART 4: Create directory using mkdir()
        printf("\n--- Creating Directory ---\n");
        /*
         * mkdir() system call:
         * Creates a new directory named "TestDirectory".
         * 0755: Read/Write/Execute for Owner; Read/Execute for Group and Others.
         */
        if (mkdir("TestDirectory", 0755) == 0)
                printf("Directory created successfully\n");
        else
                printf("Directory may already exist\n");

        // PART 5: Display directory contents using opendir() and readdir()
        printf("\n--- Directory Contents ---\n");
/* opendir() system call: Opens the current directory (".") for reading stream contents */
        dir = opendir(".");
        /* Checks if opening the directory stream failed */
        if (dir == NULL)
        {
                printf("Cannot open directory\n");
                return 1;
        }
        /* readdir() system call loop:
         * Reads directory entries sequentially.
         * Terminates when it hits the end of the directory (returns NULL).     */
        while ((entry = readdir(dir)) != NULL)
        {
                /* entry->d_name accesses the string filename of the current record */
                printf("%s\n", entry->d_name);
        }
```

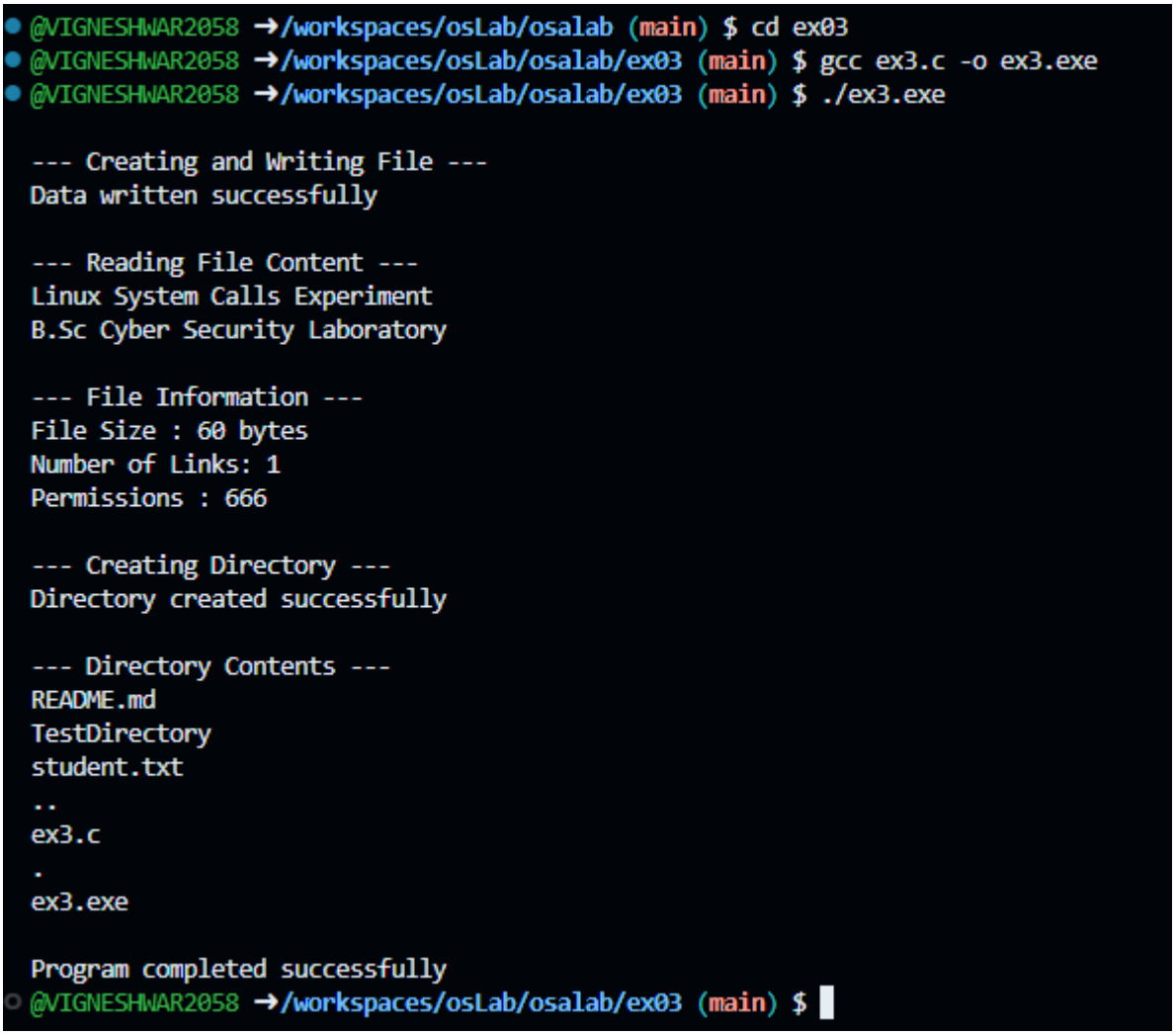
```
    closedir(dir);
    printf("\nProgram completed successfully\n");
    return 0;
}
```

HOW TO EXECUTE:

Compile (in Terminal): gcc file_operation.c -o file_operation

Run: ./file_operation

OUTPUT :



4	**Process Creation and Management Using Linux System Calls**	**30Jul2026**

AIM :

To study process creation, execution, and synchronization using Linux process system calls:

- fork()
- exec()
- wait()
- getpid()
- getppid()

and to understand parent-child process relationships.

CONTEXT:

A process is a program that is currently being executed. Linux allows a process to create another process (called a child process) and manage its execution.

Linux System Calls Used

System Call	Function
`fork()`	Creates a new child process by duplicating the current (parent) process.
`getpid()`	Returns the Process ID (PID) of the currently executing process.
`getppid()`	Returns the Process ID (PID) of the parent process.
`wait()`	Suspends the parent process until the child process terminates.
`sleep()`	Suspends the execution of the current process for the specified number of seconds. *(Library function that internally uses kernel timing services; often included in OS labs.)*

SOURCE CODE :

1. **forkDemo.c**

```
#include <stdio.h>
#include <unistd.h> // fork(), getpid(), getppid()
#include <fcntl.h>

int main()
{
        pid_t pid; // Variable to store process ID returned by fork()
        printf("Program execution started\n");
        pid = fork(); // Create a child process
        if (pid < 0) // Check whether process creation failed
        {
                printf("Unable to create child process\n");
                return 1;
        }
        //Child process block, pid value will be 0 inside child process.
        else if (pid == 0)
        {
                printf("\n--- Child Process ---\n");
                // Display child's own process ID
                printf("Child Process ID : %d\n", getpid());
                // Display parent's process ID
                printf("Parent Process ID : %d\n", getppid());
        }
        //Parent process block, pid contains the child process ID.
        else
        {
                printf("\n--- Parent Process ---\n");
```

```
            // Display parent's own process ID
            printf("Parent Process ID : %d\n", getpid());
            // Display child process ID
            printf("Child Process ID : %d\n", pid);
        }
        printf("\nProcess execution completed\n");
        return 0;
    }
```

HOW TO EXECUTE:

Compile (in Terminal): gcc fork_demo.c -o fork_demo

Run: ./fork_demo

2. waitDemo.c

```
#include <stdio.h>
#include <unistd.h> // fork(), getpid(), sleep()
#include <sys/wait.h> // wait()
int main()
{
        pid_t pid;
        pid = fork();  // Create a child process
        if (pid < 0) // Check process creation failure
        {
            printf("Process creation failed\n");
            return 1;
        }
        else if (pid == 0) //Child Process, pid == 0 indicates child execution.
        {
            printf("\n--- Child Process Started ---\n");
            printf("Child Process ID : %d\n", getpid());
```

```
            sleep(3); // Simulate some processing time
            printf("Child Process Completed\n");
        }
        else
        {
            //Parent waits for child completion.
            printf("\n--- Parent Process Started ---\n");
            printf("Parent is waiting for child process...\n");

            // wait() suspends parent execution until child process terminates.
            wait(NULL);
            printf("Child process completed\n");
            printf("Parent Process ID : %d\n", getpid());
            printf("Parent Process Resumed");
        }
        return 0;
}
```

HOW TO EXECUTE:

Compile (in Terminal): gcc wait_demo.c -o wait_demo

Run: ./wait_demo

OUTPUT :

forkDemo.c

```
@VIGNESHWAR2058 →/workspaces/osLab/osalab (main) $ cd ex04
@VIGNESHWAR2058 →/workspaces/osLab/osalab/ex04 (main) $ gcc fork_demo.c -o fork_demo.exe
@VIGNESHWAR2058 →/workspaces/osLab/osalab/ex04 (main) $ ./fork_demo.exe
Program execution started

--- Parent Process ---
Parent Process ID : 9957
Child Process ID : 9958

Process execution completed

--- Child Process ---
Child Process ID : 9958
Parent Process ID : 9957

Process execution completed
@VIGNESHWAR2058 →/workspaces/osLab/osalab/ex04 (main) $
```

waitDemo.c

```
@VIGNESHWAR2058 →/workspaces/osLab/osalab/ex04 (main) $ gcc wait_demo.c -o wait_demo.exe
@VIGNESHWAR2058 →/workspaces/osLab/osalab/ex04 (main) $ ./wait_demo.exe

--- Parent Process Started ---
Parent is waiting for child process...

--- Child Process Started ---
Child Process ID : 11306
Child Process Completed
Child process completed
Parent Process ID : 11305
Parent Process Resumed
@VIGNESHWAR2058 →/workspaces/osLab/osalab/ex04 (main) $
```

5	Signal Handling in Linux	13Aug2026

AIM :

To demonstrate inter-process communication using Linux signals and study signal generation, handling, and process synchronization using:

- signal()
- kill()
- raise()
- alarm()
- pause()

CONTEXT:

Inter-Process Communication (IPC) is the set of mechanisms that allow processes to communicate (using Shared memory, Pipes, Sockets, Signals) and synchronize with each other while running.

We explore, Signal: It is a software interrupt used by the Linux operating system to notify a process that a specific event has occurred. Signals can be generated by the operating system, the user, or another process.

Linux System Calls / Signal Functions Used

Function	Function/Purpose
`signal()`	Registers a signal handler for a specified signal.
`raise()`	Sends a signal to the currently executing process.
`getpid()`	Returns the Process ID (PID) of the current process.

Signals Used

Signal	Signal Number	Purpose
`SIGINT`	2	Interrupt signal. Normally generated when the user presses `Ctrl + C` to stop a running program.
`SIGTERM`	15	Requests a process to terminate gracefully.
`SIGKILL`	9	Immediately terminates a process and cannot be caught or ignored.
`SIGSTOP`	19	Stops a running process and cannot be caught or ignored.
`SIGCONT`	18	Continues a process that has been stopped.

SOURCE CODE :

```
#include <stdio.h>
#include <signal.h> // signal handling functions (signal, raise) and macros (SIGINT)
#include <unistd.h> // for process handling system calls like getpid()
/* Signal handler function:
 * Executes asynchronously whenever the registered signal is caught by the process.
 * 'signal_number' receives the exact ID of the triggered signal from the OS. */
void handler(int signal_number)
{
        printf("\nSignal received successfully\n");
        printf("Signal Number : %d\n", signal_number);
}
int main()
{
        //Register SIGINT handler.
        /* signal() system call:
         * Registers our custom 'handler' function to catch the SIGINT signal.
         * SIGINT is the interrupt signal typically sent by pressing Ctrl+C.     */
        signal(SIGINT, handler);

/* getpid(): Retrieves the unique Process ID (PID) of the current running program */
        printf("Current Process ID: %d\n", getpid());
        printf("Generating SIGINT using raise()\n");

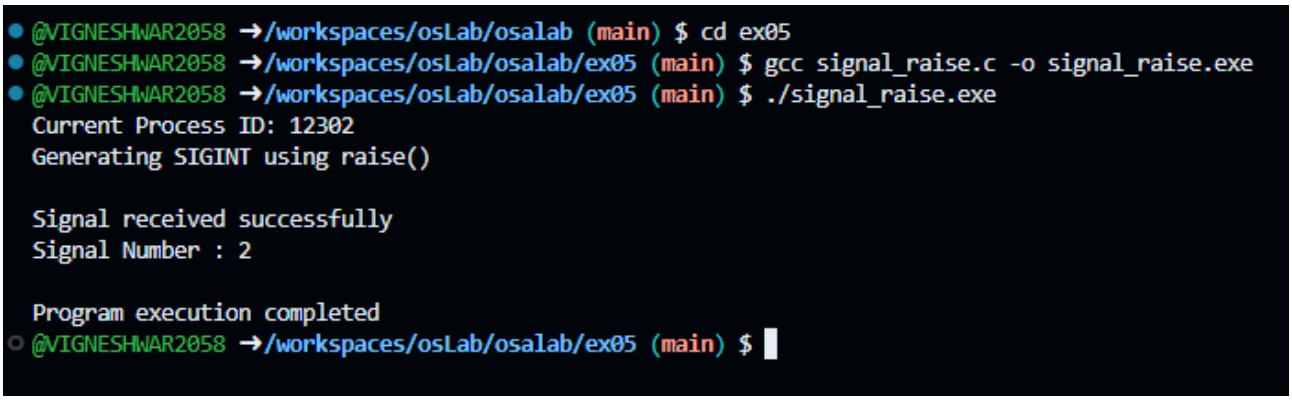
        //raise() sends SIGINT signal to the same process.
        raise(SIGINT);
        printf("\nProgram execution completed\n");
        return 0;
}
```

HOW TO EXECUTE:

Compile (in Terminal): gcc signal_raise.c -o signal_raise

Run: ./signal_raise

OUTPUT :



6	Inter-Process Communication Using Pipes and FIFO	20aug2026

AIM :

To implement inter-process communication between processes using:

1. Unnamed pipes (pipe())
2. Named pipes / FIFO (mkfifo())

and exchange data between related and unrelated processes.

CONTEXT:

This program demonstrates Inter-Process Communication (IPC) using an unnamed pipe in the Linux operating system.

We explore, Pipe: It is a communication mechanism that allows two related processes (a parent and its child) to exchange data.

Linux System Calls Used

System Call	Function
`pipe()`	Creates an unnamed pipe for communication between related processes.
`fork()`	Creates a child process from the parent process.
`read()`	Reads data from the pipe (or a file descriptor).
`write()`	Writes data to the pipe (or a file descriptor).
`close()`	Closes the read or write end of the pipe and releases resources.

SOURCE CODE :

```
#include <stdio.h>
#include <unistd.h> // Required for pipe(), fork(), read(), write(), and close()
#include <string.h>  // Required for strlen() to measure message size
#include <fcntl.h>
int main()
{
        /* fd[2] array holds two file descriptors: fd[0] for reading, fd[1] for writing */
        int fd[2];
        /* pid_t is a data type used to represent process IDs */
        pid_t pid;
        char message[] = "Hello from Parent Process";
        char buffer[100];
        // Create pipe, fd[0] -> Reading end, fd[1] -> Writing end
        if (pipe(fd) == -1)
        {
                printf("Pipe creation failed\n");
                return 1;
        }
        /* fork() system call: Clones the calling process to create a new child process.
         * Returns 0 to the child process, and the child's actual PID to the parent  */
        pid = fork();

        /* Checks if fork() failed to allocate/spawn a new process */
        if (pid < 0)
        {
                printf("Process creation failed\n");
                return 1;
        }
```

```
        // Child Process: Reads data from pipe
        else if (pid == 0)
        {
// Unused descriptors should be closed; child only reads, so it closes the write end (fd[1])
                close(fd[1]);

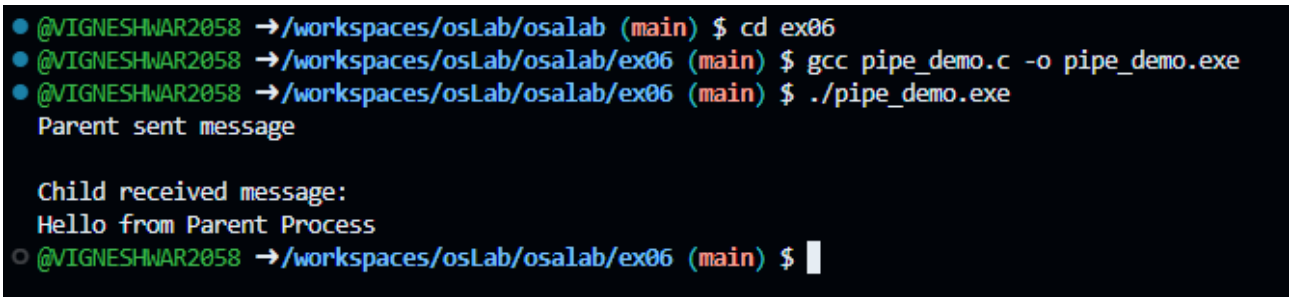
                read(fd[0], buffer, sizeof(buffer));
                printf("\nChild received message:\n");
                printf("%s\n", buffer);
                close(fd[0]);
        }
        // Parent Process: Writes data into pipe
        else
        {
                close(fd[0]);
/* write() system call: Pushes the message data string into the write end of the pipe (fd[1]).
 'strlen(message) + 1' ensures the null terminator ('\0') is also sent through the pipe. */
                write(fd[1], message, strlen(message) + 1);
                printf("Parent sent message\n");
                close(fd[1]);
        }
        return 0;
}
```

HOW TO EXECUTE:

Compile (in Terminal): gcc exp6.c -o exp6

Run: ./exp6

OUTPUT :


@VIGNESHWAR2058 →/workspaces/osLab/osalab (main) $ cd ex06
@VIGNESHWAR2058 →/workspaces/osLab/osalab/ex06 (main) $ gcc pipe_demo.c -o pipe_demo.exe
@VIGNESHWAR2058 →/workspaces/osLab/osalab/ex06 (main) $ ./pipe_demo.exe
Parent sent message
Child received message:
Hello from Parent Process
@VIGNESHWAR2058 →/workspaces/osLab/osalab/ex06 (main) $

7	Inter-Process Communication Using Shared Memory	10Sep2026

AIM :

To demonstrate inter-process communication between processes using shared memory segments and implement operations to create, access, modify, and release shared memory resources.

CONTEXT:

Inter-Process Communication (IPC) is the set of mechanisms that allow processes to communicate (using Shared memory, Pipes, Sockets, Signals) and synchronize with each other while running.

We explore, shared memory: It is one of the fastest IPC mechanisms because multiple processes can directly access the same region of memory without repeatedly transferring data between them.

Linux System Calls / Functions Used

System Call / Function	Function
`shm_open()`	Creates or opens a shared memory object.
`ftruncate()`	Sets the size of the shared memory object.
`mmap()`	Maps the shared memory object into the process's address space.
`fork()`	Creates a child process.
`wait()`	Makes the parent process wait until the child process finishes.
`munmap()`	Removes the mapped shared memory region from the process's address space.
`close()`	Closes the shared memory object descriptor.
`shm_unlink()`	Deletes the shared memory object from the system after use.
`sleep()`	Suspends execution for a specified number of seconds (used here to ensure the parent writes before the child reads).

SOURCE CODE :

```
#include<stdio.h>
#include<stdlib.h>
#include<string.h>
#include<fcntl.h>
#include<sys/mman.h>  // Memory mapping operations like shm_open( ), mmap( )
#include<sys/unistd.h>
#include<sys/types.h>
#include<sys/wait.h>  // Process waiting and synchronization wait(), waitpid(), etc
int main()
{
        // Name of the shared memory object
        const char *name = "/my_shm";

        // Size of the shared memory block (in bytes)
        const int SIZE = 4096;

        int shm_fd;  //shared memory identifier
        void *ptr;  //ptr to shared memory address

        /*      1. Create the shared memory object
                O_CREAT: Create object if it doesn't exist
                O_RDWR: Open for reading and writing
                0666: Read/Write permissions for User, Group, and Others      */

        shm_fd = shm_open(name, O_CREAT | O_RDWR, 0666);

        if (shm_fd == -1)
        {
                perror("shm_open failed");  //display an error message
```

```
        return 1;
    }

    //Configure the size of the shared memory segment
    if (ftruncate(shm_fd, SIZE) == -1)
    {
        perror("ftruncate failed");
        return 1;
    }

    //Map the shared memory object into memory
    ptr = mmap(0, SIZE, PROT_READ | PROT_WRITE, MAP_SHARED,
shm_fd, 0);
    if (ptr == MAP_FAILED)
    {
        perror("mmap failed");
        return 1;
    }

    //Fork a child process
    pid_t pid = fork();  //process id

    if (pid < 0)
    {
        perror("Fork failed");
        return 1;
    }
    else if (pid == 0)
    {
        //Child process reads the data from the shared memory segment
```

```
        sleep(1); // Simple delay to ensure the parent writes first
        printf("\nChild: Read data from shared memory: \"%s\"\n", (char *)ptr);
        munmap(ptr, SIZE); // Clean up memory mapping in child
        close(shm_fd);
    }
    else
    {
        //Parent process writes data into the shared memory segment
        const char *message = "Hello from the Shared Memory!";

        // Copying data into the memory mapped region
        sprintf(ptr, "%s", message);

        printf("Parent: Wrote data to shared memory: \"%s\"\n", message);

        // Wait for child to finish reading before cleaning up resource
        wait(NULL);

        munmap(ptr, SIZE); // Clean up and release shared memory resources

        close(shm_fd);

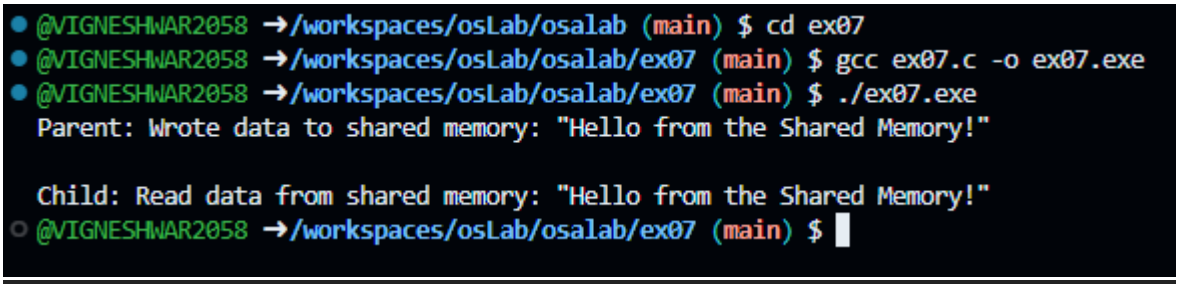
        shm_unlink(name); // Deletes the shared memory segment from system
    }
    return 0;
}
```

HOW TO EXECUTE:

Compile (in Terminal): gcc exp7.c -o exp7

Run: ./exp7

OUTPUT :



8	CPU Scheduling Algorithms	17Sep2026

AIM :

To simulate CPU scheduling algorithms (FCFS, SJF, Priority, Round Robin) and compare their performance. Students can evaluate waiting time, turnaround time, and scheduling efficiency (Process management)

CONTEXT:

This program demonstrates the First Come, First Served (FCFS) CPU scheduling algorithm, one of the simplest process scheduling algorithms used by operating systems. The process that arrives first is allocated the CPU first and runs until completion (non-preemptive scheduling).

Concepts in FCFS Algorithm

Term	Meaning	Formula
Arrival Time (AT)	Time at which a process enters the ready queue	Given as input
Burst Time (BT)	CPU time required by a process	Given as input
Start Time (ST)	Time at which the process starts execution	`ST = max(Previous CT, AT)`
Completion Time (CT)	Time at which a process finishes execution	`CT = ST + BT`
Turnaround Time (TAT)	Total time spent by a process in the system	`TAT = CT - AT`
Waiting Time (WT)	Time spent waiting in the ready queue	`WT = TAT - BT`
Average Turnaround Time	Average turnaround time of all processes	`ΣTAT / n`
Average Waiting Time	Average waiting time of all processes	`ΣWT / n`

SOURCE CODE :

```
#include <stdio.h>
struct Process  // to store process details
{
        int id;  // process id
        int arrival_time;
        int burst_time;
        int completion_time;
        int turnaround_time;
        int waiting_time;
};
int main()
{
        int n;
        float total_wt = 0, total_tat = 0;
        printf("Enter number of processes: ");
        if (scanf("%d", &n) != 1 || n <= 0)
        {
                printf("Invalid number of processes.\n");
                return 1;
        }
        struct Process p[n];
        // Read user inputs
        for (int i = 0; i < n; i++)
        {
                p[i].id = i + 1;  // assign process id
                printf("Enter Arrival Time and Burst Time for Process %d: ", p[i].id);
                scanf("%d %d", &p[i].arrival_time, &p[i].burst_time);
        }
```

```
// Sort processes based on Arrival Time (Bubble Sort)
for (int i = 0; i < n - 1; i++)
{
        for (int j = 0; j < n - i - 1; j++)
        {
                if (p[j].arrival_time > p[j + 1].arrival_time)
                {
                        struct Process temp = p[j];
                        p[j] = p[j + 1];
                        p[j + 1] = temp;
                }
        }
}

// FCFS Scheduling Calculations
int current_time = 0;
for (int i = 0; i < n; i++)
{
        // If CPU is idle waiting for the next process to arrive
        if (current_time < p[i].arrival_time)
                current_time = p[i].arrival_time;
        p[i].completion_time = current_time + p[i].burst_time;
        p[i].turnaround_time = p[i].completion_time - p[i].arrival_time;
        p[i].waiting_time = p[i].turnaround_time - p[i].burst_time;

        current_time = p[i].completion_time;

        total_wt += p[i].waiting_time;
        total_tat += p[i].turnaround_time;
}
```

```
    // Print Results Matrix
    printf("\n%-10s %-10s %-10s %-12s %-12s %-10s\n", "Process",  "Arrival",
                                    "Burst", "Completion", "Turnaround", "Waiting");

    for (int i = 0; i < n; i++)
    {
        printf("P%-9d %-10d %-10d %-12d %-12d %-10d\n",
            p[i].id,
            p[i].arrival_time,
            p[i].burst_time,
            p[i].completion_time,
            p[i].turnaround_time,
            p[i].waiting_time);
    }
    printf("\nAverage Turnaround Time = %.2f", total_tat / n);
    printf("\nAverage Waiting Time = %.2f\n", total_wt / n);

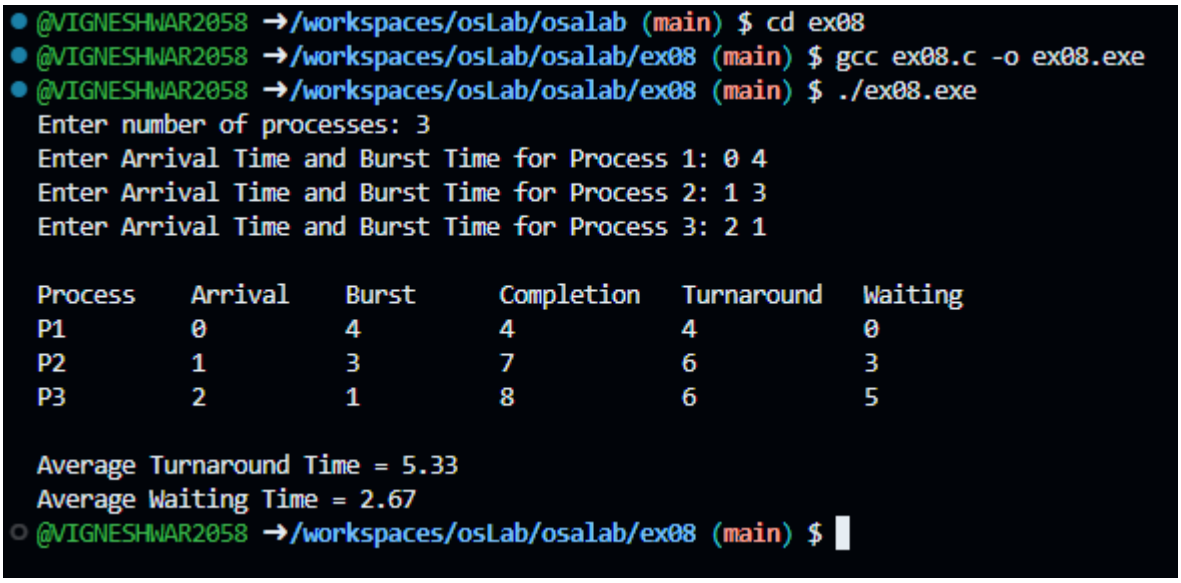
    return 0;
}
```

HOW TO EXECUTE:

Compile (in Terminal): gcc exp8.c -o exp8

Run: ./exp8

OUTPUT :


@VIGNESHWAR2058 →/workspaces/osLab/osalab (main) $ cd ex08
@VIGNESHWAR2058 →/workspaces/osLab/osalab/ex08 (main) $ gcc ex08.c -o ex08.exe
@VIGNESHWAR2058 →/workspaces/osLab/osalab/ex08 (main) $ ./ex08.exe
Enter number of processes: 3
Enter Arrival Time and Burst Time for Process 1: 0 4
Enter Arrival Time and Burst Time for Process 2: 1 3
Enter Arrival Time and Burst Time for Process 3: 2 1
Process Arrival Burst Completion Turnaround Waiting
P1 0 4 4 4 0
P2 1 3 7 6 3
P3 2 1 8 6 5
Average Turnaround Time = 5.33
Average Waiting Time = 2.67
@VIGNESHWAR2058 →/workspaces/osLab/osalab/ex08 (main) $

9	**Memory Management – Page Replacement Algorithms**	**24Sep2026**

AIM :

To simulate page replacement algorithms (FIFO, LRU, and Optimal) (Memory management)

CONTEXT:

FIFO (First-In, First-Out) Page Replacement Algorithm, which is one of the page replacement techniques used by an operating system for virtual memory management.

When the CPU requests a page that is not currently in the available memory frames, a page fault occurs. If all memory frames are occupied, the operating system must decide which page to remove to make room for the new page. The FIFO algorithm replaces the page that has been in memory the longest (the oldest page).

SOURCE CODE :

```
#include <stdio.h>
int main()
{
    // Array to store page reference string (maximum 100 pages)
    int pages[100];
    // Array to represent memory frames (maximum 50 frames)
    int frames[50];
    int n, f; // n = number of pages, f = number of frames
    int i, j; // Loop variables
    int pageFaults = 0; // Counts total page faults
    int index = 0; // Points to the next frame to be replaced (FIFO pointer)
    int found; // Flag to check whether page is already in memory

    // Read number of page references
    printf("Enter number of pages: ");
    scanf("%d", &n);

    // Read page reference string
    printf("Enter page reference string:\n");
    for (i = 0; i < n; i++)
        scanf("%d", &pages[i]);

    // Read number of available frames
    printf("Enter number of frames: ");
    scanf("%d", &f);

    // Initially all frames are empty
    for (i = 0; i < f; i++)
        frames[i] = -1;
```

```
// Display heading
printf("\nPage\tFrames\n");

// Process each page in the reference string
for (i = 0; i < n; i++)
{
        // Assume page is not present
        found = 0;

        // Search all frames to check if page already exists
        for (j = 0; j < f; j++)
        {
                // Page found in memory
                if (frames[j] == pages[i])
                {
                        found = 1;
                        break; // No need to continue searching
                }
        }

        // If page is not found, page fault occurs
        if (!found)
        {
                // Replace the oldest page using FIFO
                frames[index] = pages[i];

                // Move pointer to next frame
                index = (index + 1) % f;

                // Increase page fault count
```

```
                pageFaults++;
            }

            // Print current page
            printf("%d\t", pages[i]);

            // Display contents of all frames
            for (j = 0; j < f; j++)
            {
                // Empty frame
                if (frames[j] == -1)
                    printf("- ");

                // Occupied frame
                else
                    printf("%d ", frames[j]);
            }
            // Display if page fault occurred
            if (!found)
                printf("(Page Fault)");

            printf("\n");
        }
        // Display final statistics
        printf("\nTotal Page Faults = %d\n", pageFaults);
        // Hits = Total Pages - Page Faults
        printf("Total Page Hits = %d\n", n - pageFaults);

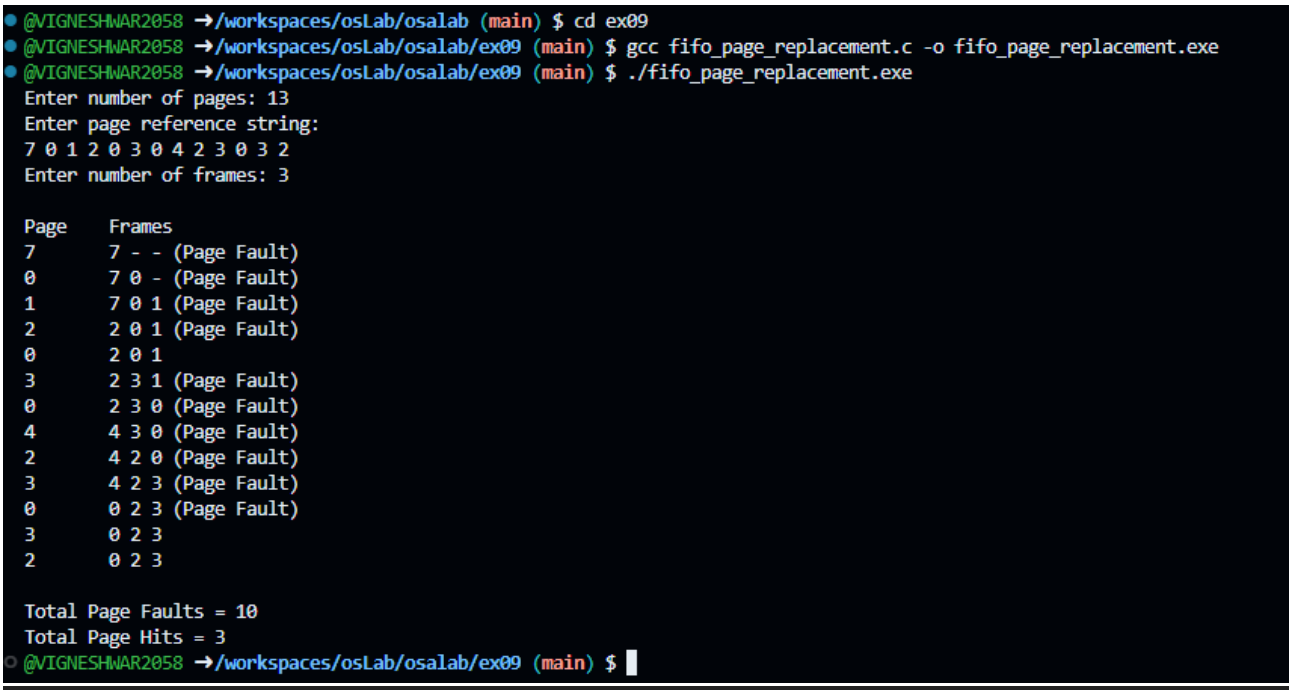
        return 0;
}
```

HOW TO EXECUTE:

Compile (in Terminal): gcc exp9.c -o exp9

Run: ./exp9

OUTPUT :



10	Linux File Permissions and File Information	01Oct2026

AIM :

To study Linux file permissions and retrieve file metadata using stat() and related system calls.

CONTEXT:

Metadata: Metadata is data that describes another piece of data.

This program demonstrates how to retrieve and display file metadata in the Linux operating system using the stat() system call. It accepts a filename from the user, obtains information about the file, and displays following details:

Metadata Field	Explanation
File Size	The total size of the file measured in bytes.
Inode Number	A unique number that identifies a file in the Linux file system.
Owner	The user who owns or created the file.
Group	The group of users associated with the file.
Permissions	Specifies who can read, write, or execute the file.
Number of Hard Links	The number of directory entries that point to the same file.
File Timestamps	Records when the file was last accessed, modified, or its metadata was changed.

SOURCE CODE :

```
#include <stdio.h>
#include <stdlib.h>
#include <sys/stat.h> // stat(), struct stat
#include <unistd.h> // POSIX functions
#include <pwd.h> // getpwuid()
#include <grp.h> // getgrgid()
#include <time.h> // ctime()

int main()
{
    // Variable to store file information
    struct stat fileStat;

    // Array to store the filename entered by the user
    char filename[100];

    // Read filename
    printf("Enter filename: ");
    scanf("%s", filename);

    // Get file information
    if (stat(filename, &fileStat) == -1)
    {
        perror("Error");
        return 1;
    }

    printf("\n===== File Information =====\n");
```

```
// File size in bytes
printf("File Size : %ld bytes\n", fileStat.st_size);

// Inode number
printf("Inode Number : %ld\n", fileStat.st_ino);

// Number of hard links
printf("Hard Links : %ld\n", fileStat.st_nlink);

// User ID (Owner)
printf("Owner UID : %d\n", fileStat.st_uid);

// Group ID
printf("Group GID : %d\n", fileStat.st_gid);

// Owner name
printf("Owner Name : %s\n", getpwuid(fileStat.st_uid)->pw_name);

// Group name
printf("Group Name : %s\n", getgrgid(fileStat.st_gid)->gr_name);

// File permissions
printf("\nPermissions : ");

printf((S_ISDIR(fileStat.st_mode)) ? "d" : "-");

printf((fileStat.st_mode & S_IRUSR) ? "r" : "-");
printf((fileStat.st_mode & S_IWUSR) ? "w" : "-");
printf((fileStat.st_mode & S_IXUSR) ? "x" : "-");
```

```
        printf((fileStat.st_mode & S_IRGRP) ? "r" : "-");
        printf((fileStat.st_mode & S_IWGRP) ? "w" : "-");
        printf((fileStat.st_mode & S_IXGRP) ? "x" : "-");

        printf((fileStat.st_mode & S_IROTH) ? "r" : "-");
        printf((fileStat.st_mode & S_IWOTH) ? "w" : "-");
        printf((fileStat.st_mode & S_IXOTH) ? "x" : "-");

        printf("\n");

        // Last access time
        printf("Last Access : %s", ctime(&fileStat.st_atime));

        // Last modification time
        printf("Last Modification : %s", ctime(&fileStat.st_mtime));

        // Last status change
        printf("Last Status Change : %s", ctime(&fileStat.st_ctime));

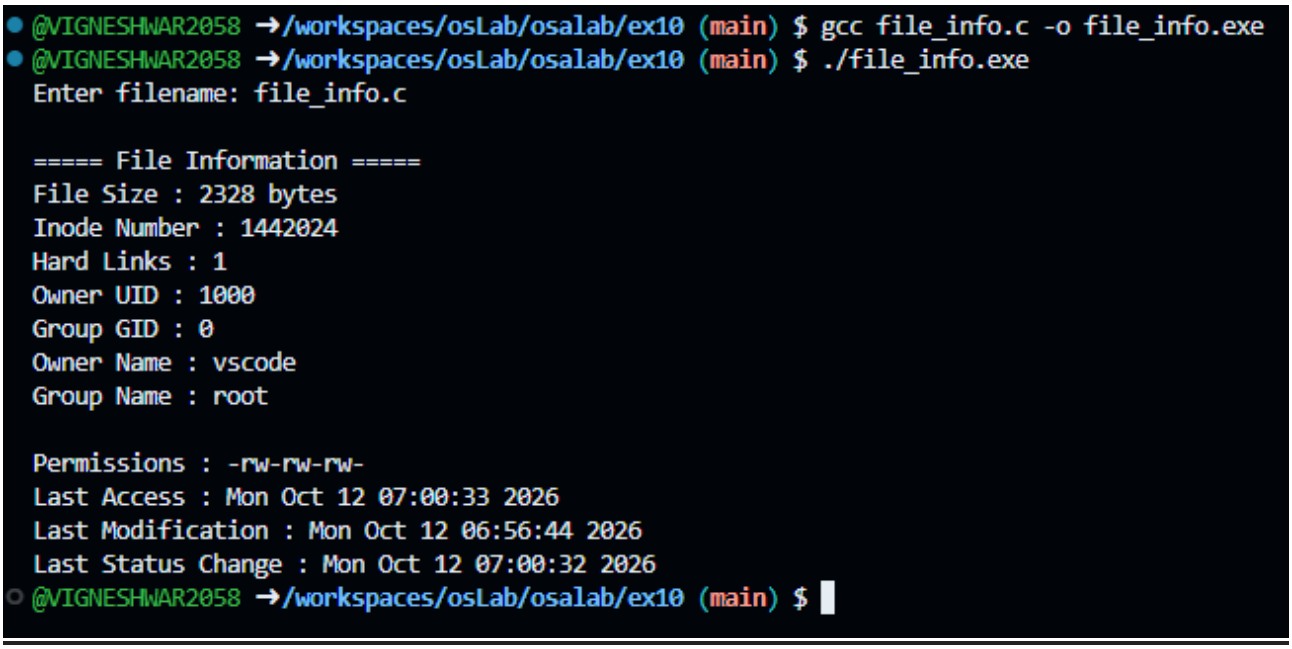
        return 0;
}
```

HOW TO EXECUTE:

Compile (in Terminal): gcc file_info.c -o file_info

Run: ./file_info

OUTPUT :


@VIGNESHWAR2058 →/workspaces/osLab/osalab/ex10 (main) $ gcc file_info.c -o file_info.exe
@VIGNESHWAR2058 →/workspaces/osLab/osalab/ex10 (main) $ ./file_info.exe
Enter filename: file_info.c
===== File Information =====
File Size : 2328 bytes
Inode Number : 1442024
Hard Links : 1
Owner UID : 1000
Group GID : 0
Owner Name : vscode
Group Name : root
Permissions : -rw-rw-rw-
Last Access : Mon Oct 12 07:00:33 2026
Last Modification : Mon Oct 12 06:56:44 2026
Last Status Change : Mon Oct 12 07:00:32 2026
@VIGNESHWAR2058 →/workspaces/osLab/osalab/ex10 (main) $